UNDEF

FADENA
Facultad de Defensa Nacional

Facultad de Defensa Nacional

Carrera: Lic. en Ciberdefensa

Materia

Sistemas Operativos I

Unidad 4 / Semana 11

Año: 2025

Contenidos

Administración de Memoria. Memoria virtual. Paginación. Segmentación.

Índice

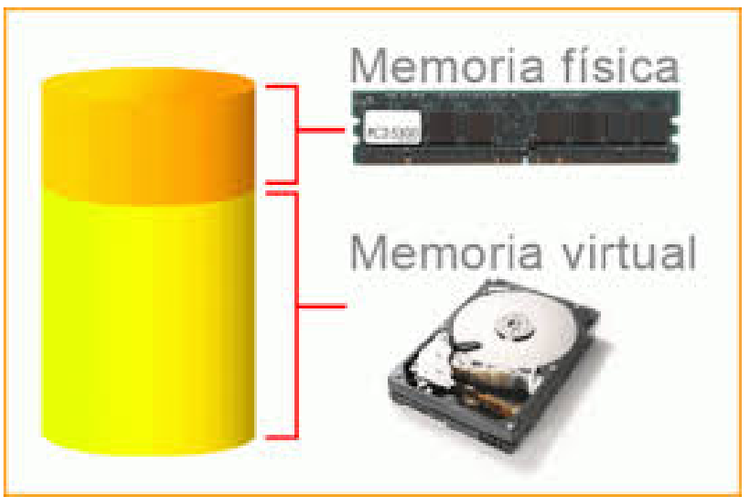
1. Memoria virtual **3**
1.1 Memoria virtual 3
1.1.1 Abstracción de memoria 3
1.1.2 Mecanismo de administración de la memoria 4
1.1.3 Uso de la memoria secundaria como extensión de la memoria principal 4
2. Paginación **5**
2.1 Traducción de direcciones lógicas a físicas 7
2.2 Estructura de la tabla de páginas 8
2.2.1 Tabla de un solo nivel 9
2.2.2 Tabla de varios niveles 10
2.2.3 Tabla invertida 10
2.3 Buffer de traducción anticipada (TLB) 12
2.4 Fallo de página 13
2.5 Algoritmos de reemplazo de páginas 13
2.5.1 Óptimo. 14
2.5.2 Usado menos recientemente (least recently used-LRU). 14
2.5.3 FIFO (first-in-first-out). 14
2.5.4 Reloj. 14
2.6 Thrashing 14
3. Segmentación **15**
3.1 Segmentación Paginada 17

1. Memoria virtual

1.1 Memoria virtual

El término memoria virtual se asocia a dos conceptos que normalmente aparecen unidos: el uso de almacenamiento secundario para ofrecer al conjunto de las aplicaciones la ilusión de tener más memoria RAM de la que realmente hay en el sistema.

Esta ilusión existe a nivel del sistema, es decir, teniendo en ejecución más aplicaciones de las que realmente caben en la memoria principal, sin que por ello cada aplicación individual pueda usar más memoria de la que realmente hay o incluso de forma más general, ofreciendo a cada aplicación más memoria de la que existe físicamente en la máquina.



Ofrecer a las aplicaciones la ilusión de que están solas en el sistema, y que por lo tanto, pueden usar el espacio de direcciones completo. Esta técnica facilita enormemente la generación de código, puesto que el compilador no tiene por qué preocuparse sobre dónde residirá la aplicación cuando se ejecute.

Su funcionamiento se basa en conceptos fundamentales:

1.1.1 Abstracción de memoria

La abstracción de memoria en los sistemas operativos permite a las aplicaciones trabajar con una visión simplificada y uniforme de la memoria, independientemente de cómo se gestiona realmente el hardware (direcciones reales). Esto se traduce en que cada proceso percibe que tiene acceso a un espacio de direcciones continuo, sin tener que lidiar con las complejidades de la memoria física real.

Cada proceso se ejecuta en un entorno aislado con su propio espacio de direcciones virtuales. Esto significa que, aunque físicamente la memoria pueda estar fragmentada o compartida, cada aplicación ve una memoria contigua que comienza, por ejemplo, en la dirección 0. Por detrás, el sistema operativo

utiliza técnicas como la traducción de direcciones para mapear estas direcciones virtuales a ubicaciones físicas reales.

1.1.2 Mecanismo de administración de la memoria

Para manejar esta abstracción, se utilizan mecanismos como la **paginación** y la **segmentación**. La paginación divide el espacio de direcciones en bloques (páginas) de tamaño fijo, y a su vez, la memoria física se divide en marcos. El sistema operativo mantiene una tabla de páginas que asocia cada página virtual con un marco físico. La segmentación, por otro lado, divide la memoria en segmentos de tamaño variable, organizando lógicamente el código, los datos y la pila de cada proceso.

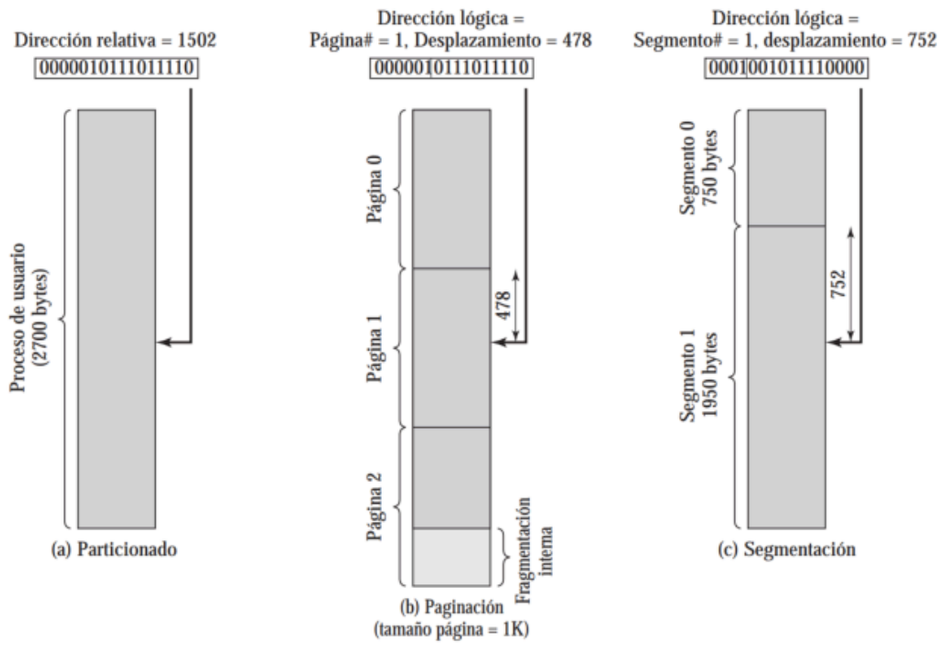
1.1.3 Uso de la memoria secundaria como extensión de la memoria principal

Cuando se ejecuta una aplicación, el sistema operativo asigna a esa aplicación un espacio de direcciones virtuales. Aunque la aplicación crea que está operando en un bloque continuo de memoria (por ejemplo, de 0 a 2 GB), el sistema operativo traduce internamente estas direcciones a fragmentos dispersos en la memoria física.

Así, si una parte de la aplicación no se está utilizando activamente, el sistema puede optar por mover esos datos al almacenamiento secundario (disco duro o SSD), liberando recursos para otras tareas sin que la aplicación perciba ninguna diferencia.

Con esto se logran 2 importantes beneficios,

1. Podemos tener en memoria más aplicaciones corriendo de las que cabrían en la memoria principal
2. Podemos ejecutar aplicaciones que utilizan más memoria de la que actualmente dispone el sistema.



Direcciones lógicas

2. Paginación

Se ha visto que el uso de particiones de tamaño fijo o variable resultaron ineficientes para el administrar la memoria, por tener fragmentación interna uno y externa el otro.

Imaginemos que dividimos la memoria en porciones de tamaño fijo relativamente pequeños y que cada proceso también lo dividimos en pequeñas partes del mismo tamaño.

A dichas porciones del *proceso* las denominaremos como **páginas** y se les asigna porciones disponibles de memoria, conocidas como **marcos**, o **marcos de páginas**. En este caso, la memoria desaprovechada por cada proceso debido a la ***fragmentación interna*** corresponde sólo a **una fracción de la última página de un proceso** y, en cambio, **no existe *fragmentación externa***.

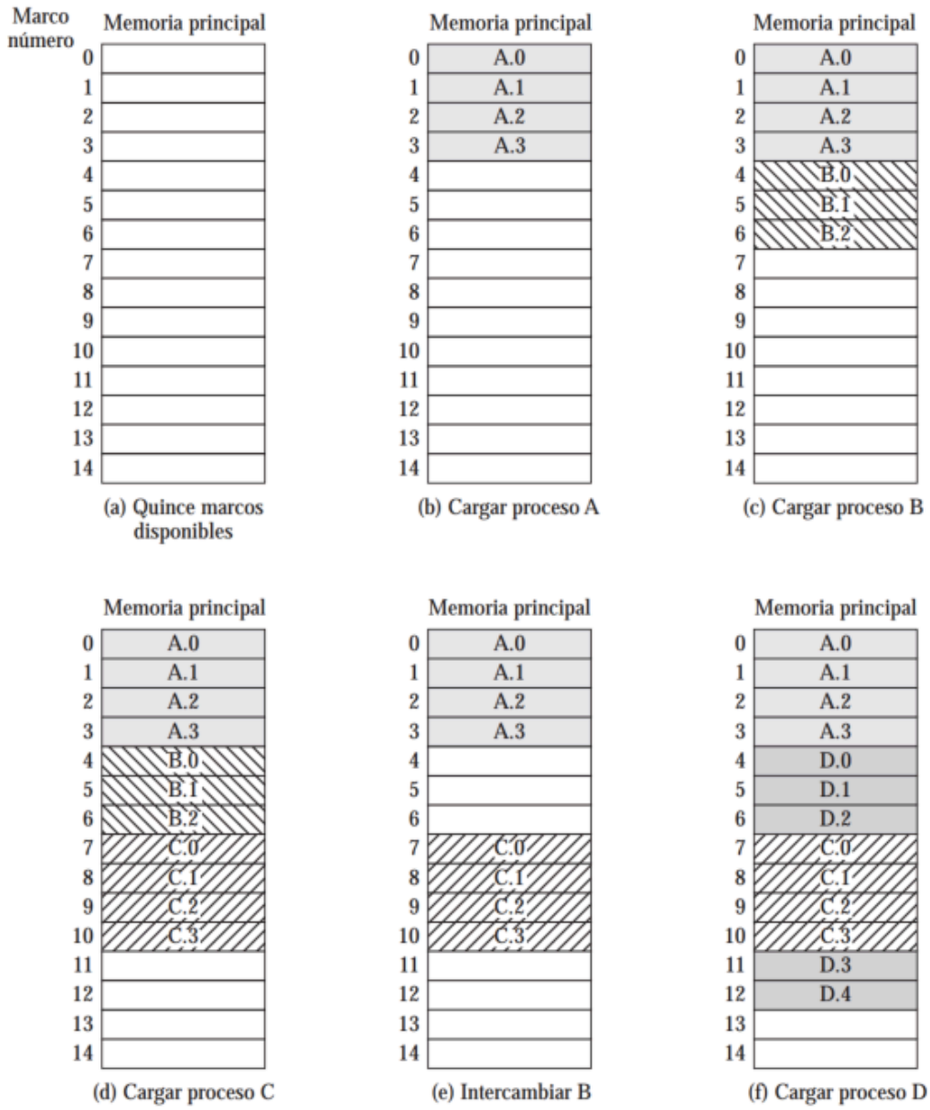


Figura 1. Asignación de páginas de procesos a marcos libres[1]

En la figura 1 se ilustra el uso de páginas y marcos en diferentes casos.

a. Se inicia con 15 marcos disponibles, identificados del 0 al 14

b. Se carga el proceso A que insume 4 marcos, se utilizan las páginas 0 a 3

c. Se carga el proceso B que tiene 3 páginas a continuación, en los marcos 4 a 6

d. Se carga el proceso C que necesita 4 marcos, se almacena en el marco 7 al 10

e. Se intercambia el proceso B (va al disco), liberando la memoria

f. Se carga el proceso D que tiene 5 páginas, se usan los marcos 4 al 6 y 11 y 12

[1] Stallings, William (2005), Sistemas operativos. Aspectos internos y principios de diseño, Quinta edición.

Vemos que la paginación tiene cierta similitud con el *particionamiento fijo*, pero tiene diferencias importantes:,

1. los marcos son particiones bastante más pequeñas (y de igual tamaño)
2. un programa podría ocupar más de una partición
3. No necesitan ser contiguas

Para administrar la paginación, el sistema operativo mantiene una **tabla de páginas** por cada proceso.

La tabla de páginas muestra la ubicación del marco por cada página del proceso además de otros datos que examinaremos más adelante. Dentro del programa, cada *dirección lógica* está formada por un **número de página** y un **desplazamiento** dentro de la página. Por conveniencia, siempre el tamaño del marco será una potencia de 2.

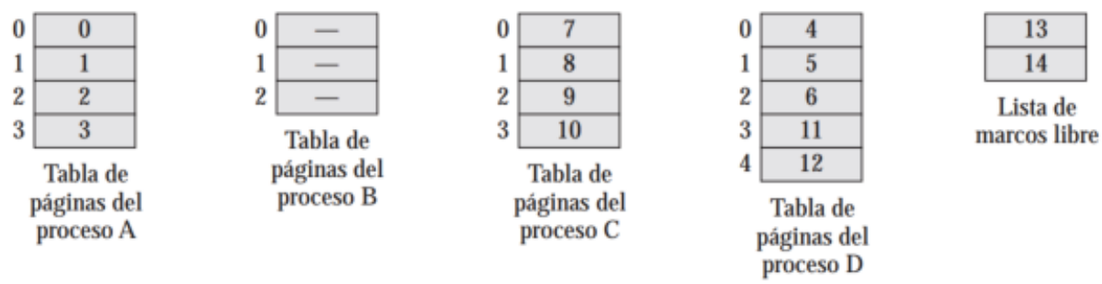


Figura 2. Estado de las tablas al momento f.[2]

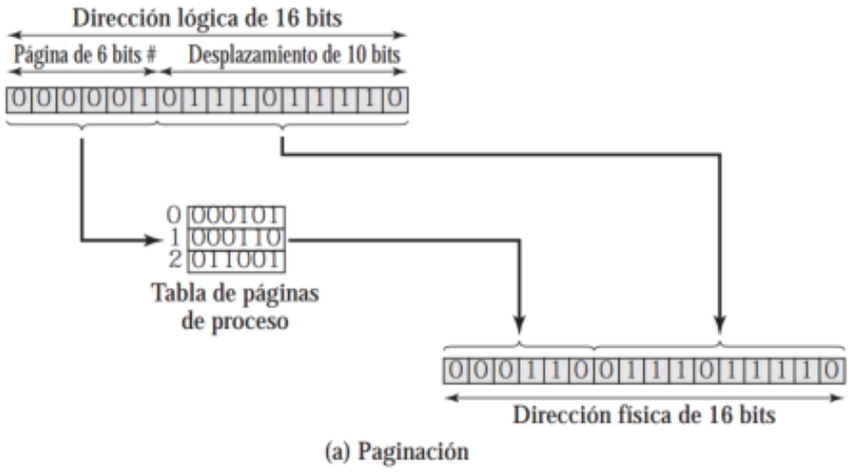
2.1 Traducción de direcciones lógicas a físicas

Es importante recordar que en el caso de una partición, una dirección lógica es la ubicación de una palabra o ubicación de memoria relativa al comienzo del programa; el procesador la traduce en una dirección física.

Con *paginación*, la traducción de direcciones lógicas a físicas las realiza también el hardware del procesador. Ahora el procesador debe conocer cómo acceder a la tabla de páginas del proceso actual. Usando una dirección lógica que se compone de un *número de página, desplazamiento*, el procesador utiliza la tabla de páginas para producir una dirección física (*número de marco, desplazamiento*).

[2] Stallings, William (2005), Sistemas operativos. Aspectos internos y principios de diseño, Quinta edición.

1. Toma la dirección de memoria y obtiene el número de página
2. Accede a la tabla de páginas y obtiene la dirección física del inicio del marco
3. Aplica a la dirección obtenida el desplazamiento (offset) para obtener la dirección de memoria física a acceder



(a) Paginación

2.2 Estructura de la tabla de páginas

La tabla de páginas es una estructura de datos utilizada por el sistema operativo para mapear las direcciones virtuales de un proceso a direcciones físicas en la memoria real. A continuación, se describe detalladamente la estructura y los elementos que componen una tabla de páginas:

Componentes de una Entrada de Tabla de Páginas (PTE)

Cada entrada en la tabla de páginas, conocida como PTE (Page Table Entry), típicamente contiene los siguientes campos:

- Número de marco (Frame Number):

Este campo indica el número o dirección base del marco físico en el que se encuentra almacenada la página. Es el vínculo directo entre la dirección virtual y la localización en la memoria real.

- Bit de presencia (Present Bit):

Un indicador que muestra si la página está actualmente cargada en la memoria física. Si este bit no está activo, se genera un *fallo de página* al intentar acceder a la dirección virtual.

- Bit de acceso (Accessed Bit):

Este bit señala si la página ha sido recientemente leída o escrita. Es esencial para algoritmos de reemplazo contar con esta información.

- Bit de modificado o sucio (Dirty Bit):

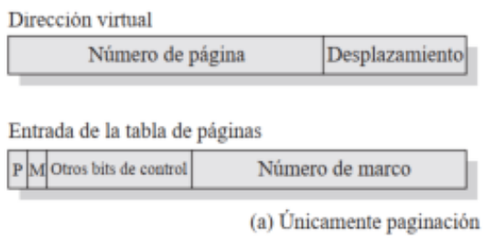
Indica si la página ha sido modificada después de haber sido cargada en memoria. Esto permite que, al necesitar reubicar o liberar la página, el sistema sepa si debe volcar la información de vuelta al almacenamiento secundario.

- Bits de permisos (Access Control Bits):

Se incluyen flags que definen los permisos de la página, tales como si se puede leer, escribir o ejecutar. Esto es fundamental para la protección del espacio de memoria de cada proceso y para evitar que un proceso acceda a memoria que no le corresponde.

- Otros campos adicionales:

Dependiendo de la arquitectura y el sistema operativo, pueden existir campos para gestionar la caché, bits de validación o indicadores especializados para optimizar el rendimiento del recorrido de páginas.



Ejemplo de tabla de páginas

2.2.1 Tabla de un solo nivel

Se organiza como un arreglo lineal en el que cada entrada corresponde a una página del espacio de direcciones virtuales. Es simple, pero en sistemas con espacios grandes (por ejemplo, direcciones de 32 o 64 bits) resulta ineficiente en cuanto a uso de memoria, ya que requiere una entrada para cada página posible, incluso si no todas se usan

2.2.2 Tabla de varios niveles

Para manejar espacios de direcciones grandes, se dividen jerárquicamente. Por ejemplo, en una estructura de dos (o más) niveles, la dirección virtual se divide en dos partes:

La primera parte indexa un directorio de páginas (primer nivel).

La segunda parte indexa la tabla de páginas específica (segundo nivel) que contiene las entradas de las páginas.

Esto permite que solo se carguen en memoria las partes del espacio que realmente se usan, optimizando el consumo de recursos. Dependiendo de la arquitectura, pueden existir tres o más niveles de tablas

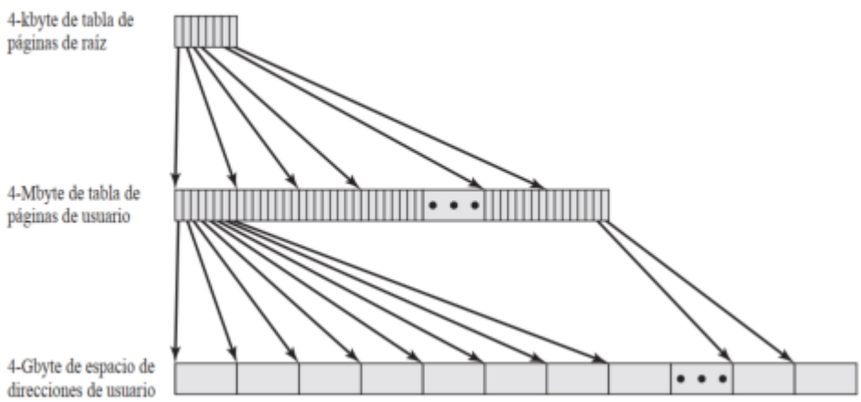


Tabla jerárquica de dos niveles

2.2.3 Tabla invertida

Una desventaja del tipo de tablas de páginas que hemos visto es que su tamaño es proporcional al espacio de direcciones virtuales.

Una estrategia alternativa al uso de tablas de páginas de uno o varios niveles es el uso de la estructura de tabla de ***páginas invertida***.

Variaciones de esta estrategia se han usado en arquitecturas como PowerPC, UltraSPARC, e IA-64.

En esta estrategia, la parte correspondiente al número de página de la dirección virtual se referencia por medio de un valor *hash* usando una función hash sencilla.

El valor hash es un puntero para la tabla de páginas invertida, que contiene las entradas de tablas de página.

Hay una entrada en la tabla de páginas invertida por cada marco de página real en lugar de uno por cada página virtual. De esta forma, lo único que se requiere para estas tablas de página siempre es una proporción fija de la memoria real, independientemente del número de procesos o de las páginas virtuales soportadas.

Debido a que más de una dirección virtual *puede traducirse en la misma entrada de la tabla hash*, una técnica de encadenamiento se utiliza para gestionar el desbordamiento. Las técnicas de hashing proporcionan habitualmente cadenas que no son excesivamente largas —entre una y dos entradas. La estructura de la tabla de páginas se denomina invertida debido a que se indexan sus entradas de la tabla de páginas ***por el número de marco*** en lugar de por el número de página virtual.

Para un tamaño de memoria física de 2^m marcos, la tabla de páginas invertida contiene 2^m entradas, de forma que la entrada en la posición i-ésima se refiere al marco i.

La entrada en la tabla de páginas incluye la siguiente información:

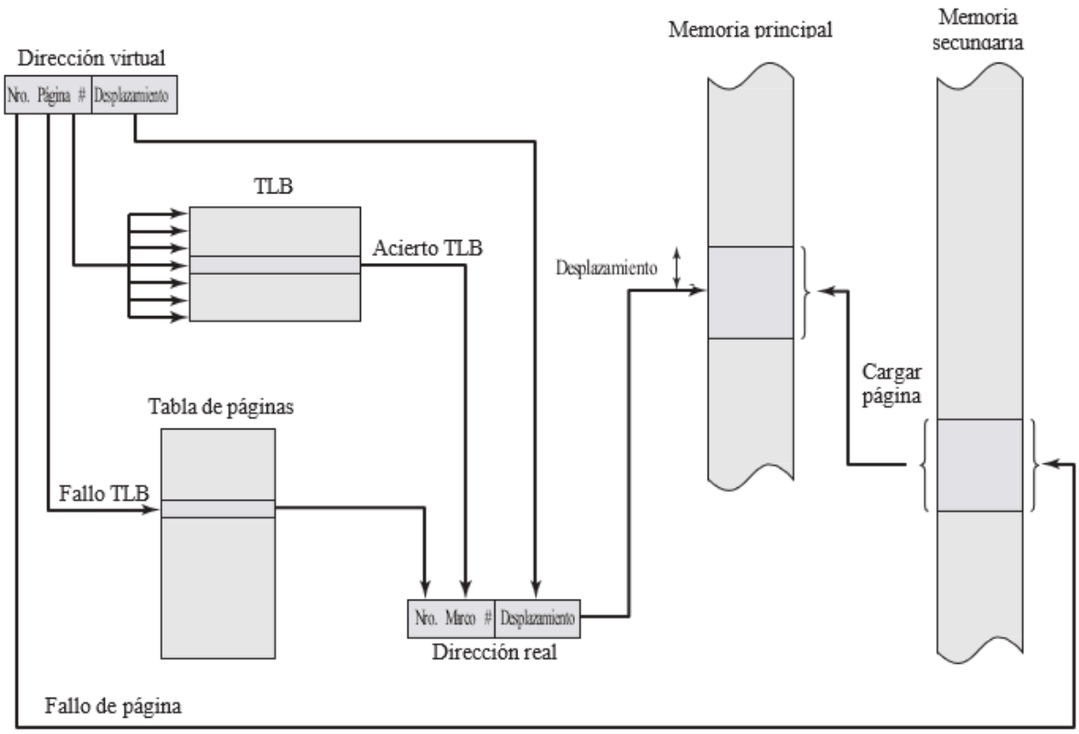
- **Número de página.** Esta es la parte correspondiente al número de página de la dirección virtual.
- **Identificador del proceso**. El proceso que es propietario de está página. La combinación de número de página e identificador del proceso identifica a una página dentro del espacio de direcciones virtuales de un proceso en particular.
- **Bits de control.** Este campo incluye los flags, como por ejemplo, válido, referenciado, y modificado; e información de protección y cerrojos.
- **Puntero de la cadena**. Este campo es nulo (indicado posiblemente por un bit adicional) si no hay más entradas encadenadas en esta entrada. En otro caso, este campo contiene el valor del índice (número entre 0 y 2^m-1) de la siguiente entrada de la cadena y se repetirá la operación hasta hallar el nro de página buscado.

2.3 Buffer de traducción anticipada (TLB)

En principio, toda referencia a la memoria virtual puede causar dos accesos a memoria física: uno para buscar la entrada en la tabla de páginas apropiada y otro para buscar los datos solicitados. De esa forma, un esquema de memoria virtual básico causaría el efecto de duplicar el tiempo de acceso a la memoria. Para solventar este problema, la mayoría de esquemas de la memoria virtual utilizan una caché especial de alta velocidad para las entradas de la tabla de página, habitualmente denominada ***buffer de traducción anticipada*** (*translation lookaside buffer* - **TLB)**.

Esta caché funciona de forma similar a una memoria caché general y contiene aquellas entradas de la tabla de páginas que han sido usadas de forma más reciente.

Dada una dirección virtual, el procesador primero examina la TLB, si la entrada de la tabla de páginas solicitada está presente (*acierto en TLB*), entonces se recupera el número de marco y se construye la dirección real. Si la entrada de la tabla de páginas solicitada no se encuentra (*fallo en la TLB*), el procesador utiliza el número de página para indexar la tabla de páginas del proceso y examinar la correspondiente entrada de la tabla de páginas. Si el bit de presente está puesto a 1, entonces la página se encuentra en memoria principal, y el procesador puede recuperar el número de marco desde la entrada de la tabla de páginas para construir la dirección real. El procesador también autorizará la TLB para incluir esta nueva entrada de tabla de páginas. Finalmente, si el bit presente no está puesto a 1, entonces la página solicitada no se encuentra en la memoria principal y se produce un fallo de acceso memoria, llamado fallo de página. En este punto, abandonamos el dominio del hardware para invocar al sistema operativo, el cual cargará la página necesaria y actualizada de la tabla de páginas.



Uso de TLB

2.4 Fallo de página

En un mundo ideal todas las páginas están cargadas en memoria y no tendríamos problemas para accederlas. Pero en realidad sabemos que esto no sucede.

Cuando un proceso utiliza direcciones virtuales para acceder a datos, el sistema operativo traduce esta dirección a una localización física de la memoria. Si la página correspondiente no ha sido cargada en la RAM —ya sea porque nunca se ha utilizado o fue desplazada a almacenamiento secundario (por ejemplo, a un archivo de intercambio o swap)— se produce un fallo de página. En ese momento, se genera una interrupción o "trap", y el sistema operativo interviene para atender la situación, gestionando la operación de E/S para obtener esa página.

2.5 Algoritmos de reemplazo de páginas

Cuando no existen marcos libres en memoria y se debe cargar una nueva página en memoria, existen diferentes algoritmos básicos que se utilizan para la selección de la página a reemplazar(víctima).

Los principales algoritmos conocidos son:

2.5.1 Óptimo.

Este algoritmo tomará como reemplazo a la página para la cuál el instante de la siguiente referencia se encuentra más lejos. Es un enfoque teórico, porque evidentemente, esta política es imposible de implementar, dado que requiere que el sistema operativo tenga un perfecto conocimiento de los eventos futuros. Sin embargo se utiliza como un estándar a partir del cual contrastar algoritmos reales.

2.5.2 Usado menos recientemente (least recently used-LRU).

Este algoritmo elige al marco de página que no ha sido utilizado durante el mayor período de tiempo. Es bastante parecida al óptimo, produciendo resultados parecidos en virtud al principio de proximidad, pero es costoso de implementar al mantener la información necesaria para identificar a la página a reemplazar

2.5.3 FIFO (first-in-first-out).

Este algoritmo reemplaza la página que ha estado en memoria durante más tiempo, (la primera en llegar) basándose en una estructura de cola. Se respeta el orden de llegada. Es sencillo de implementar, se hace sobre un buffer circular pero a veces puede expulsar páginas que se pueden necesitar en breve.

2.5.4 Reloj.

Es una mejora del algoritmo FIFO que incorpora un bit de referencia para dar “una segunda oportunidad” a las páginas que han sido usadas recientemente. Cuando se carga por primera vez se coloca esa marca en uno. Si se usa o referencia esa página, se coloca en 1 nuevamente.
Al momento de reemplazar una página, se recorre la lista, a partir de la última posición usada, si la página tiene marca de usada (bit =1) se cambia a cero y se verifica la siguiente hasta encontrar una con bit de uso en cero.

2.6 Thrashing

El thrashing en paginación se produce cuando el sistema gasta tanto tiempo gestionando fallos de página, es decir descargando y cargando páginas entre

la RAM y el área de SWAP que degrada la performance del sistema y apenas tiene tiempo para ejecutar los procesos en memoria.

Cada proceso necesita un número mínimo de páginas para operar eficazmente que se conoce como **conjunto de trabajo o working-set.**

Si el sistema gestiona incorrectamente el conjunto de trabajo o tiene un exceso del grado de multiprogramación, es decir que tiene más procesos que los puede almacenar en la memoria, aumentarán los fallos de página y la degradación del sistema producirá el thrashing.

3. Segmentación

Un programa de usuario se puede subdividir utilizando segmentación, en la cual el programa y sus datos asociados se dividen en un número de segmentos. No se requiere que todos los programas sean de la misma longitud, aunque hay una longitud máxima de segmento. Como en el caso de la paginación, una dirección lógica utilizando segmentación está compuesta por dos partes, en este caso un número de segmento y un desplazamiento.

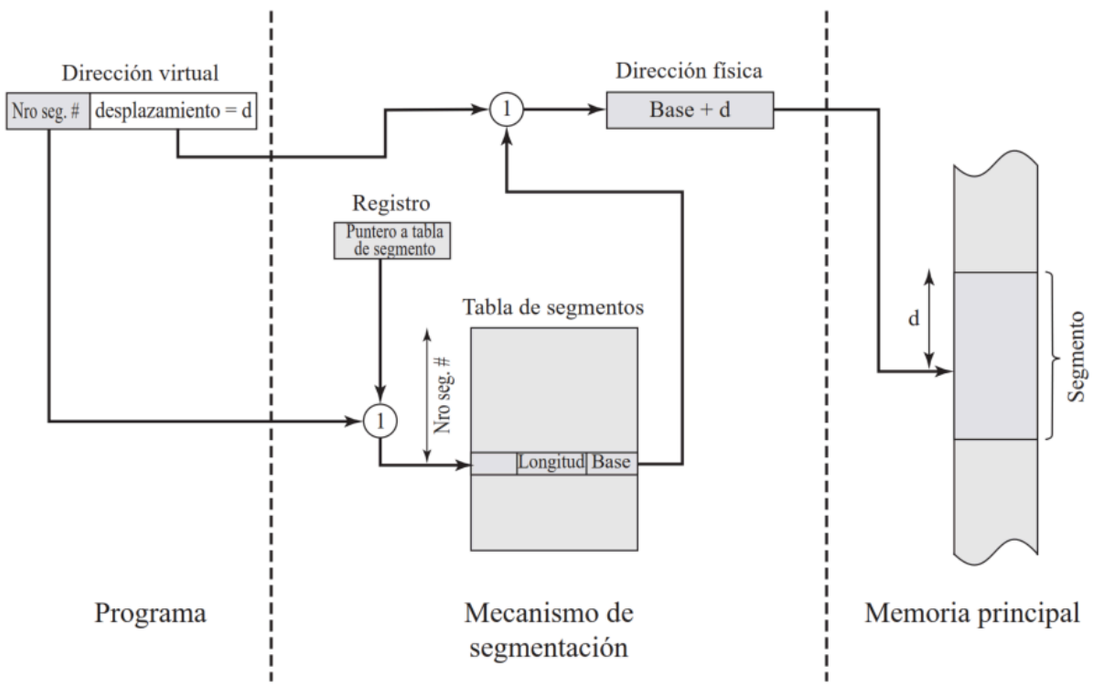
Debido al uso de segmentos de distinto tamaño, la segmentación es similar al particionamiento dinámico. En la ausencia de un esquema de *overlays* o el uso de la memoria virtual, se necesitaría que todos los segmentos de un programa se cargaran en la memoria para su ejecución. La diferencia, comparada con el particionamiento dinámico, es que con la segmentación un programa podría ocupar más de una partición, y estas particiones no necesitan ser contiguas. La segmentación ***elimina la fragmentación interna*** pero, al igual que el particionamiento dinámico, ***sufre de fragmentación externa***. Sin embargo, debido a que el proceso se divide en varias piezas más pequeñas, la fragmentación externa debería ser menor.

Mientras que la ***paginación es invisible al programador,*** la ***segmentación es normalmente visible*** y se proporciona como una utilidad para organizar programas y datos. Normalmente, el programador o compilador asignará programas y datos a diferentes segmentos. Para los propósitos de la programación modular, los programas o datos se pueden dividir posteriormente en múltiples segmentos.

El inconveniente principal de este servicio es que el programador debe ser consciente de la limitación de tamaño de segmento máximo.

Otra consecuencia de utilizar segmentos de distinto tamaño es que no hay una relación simple entre direcciones lógicas y direcciones físicas. De forma análoga a la paginación, un esquema de segmentación sencillo haría uso de una tabla de segmentos por cada proceso y una lista de bloques libre de memoria principal. Cada entrada de la tabla de segmentos tendría que proporcionar la dirección inicial de la memoria principal del correspondiente segmento. La entrada también **debería proporcionar la longitud del segmento, para asegurar que no se utilizan direcciones no válidas**. Cuando un proceso entra en el estado *Ejecutando*, la dirección de su tabla de segmentos se carga en un registro especial utilizado por el hardware de gestión de la memoria.

En síntesis, con la segmentación simple, un proceso se divide en un conjunto de segmentos que no tienen que ser del mismo tamaño. Cuando un proceso se trae a memoria, todos sus segmentos se cargan en regiones de memoria disponibles, y se crea la tabla de segmentos.



Traducción de direcciones en un sistema con segmentación

3.1 Segmentación Paginada

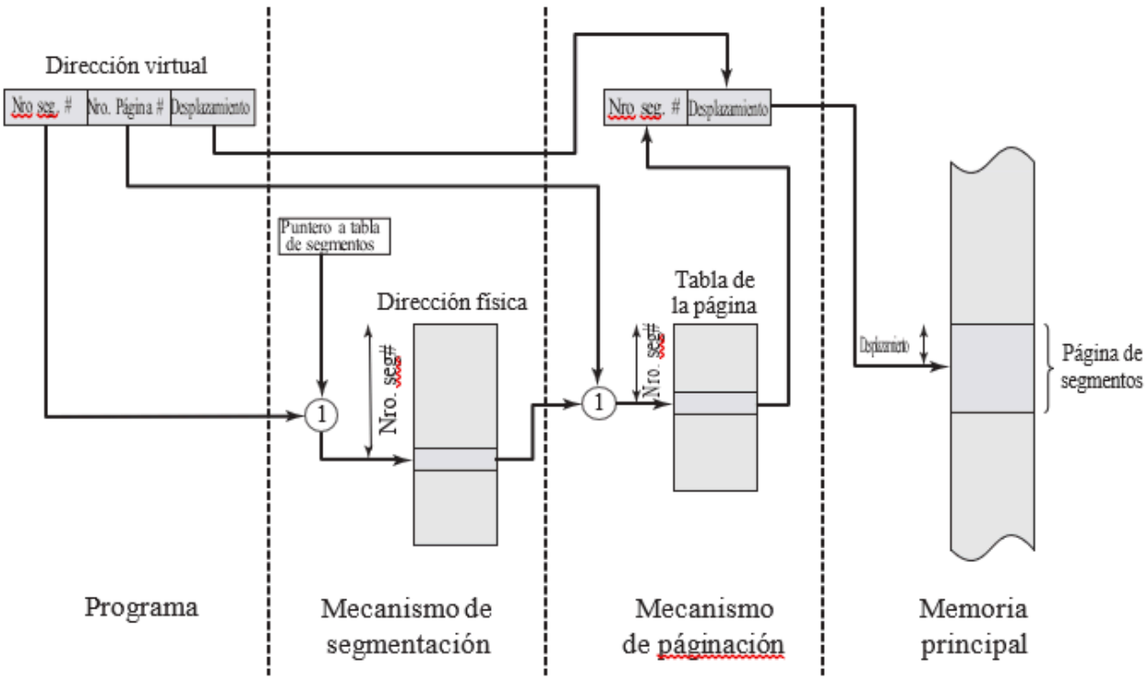
Paginación y segmentación, cada una tiene sus propias ventajas. La paginación es transparente al programador y elimina la fragmentación externa, y por tanto proporciona un uso eficiente de la memoria principal. Adicionalmente, debido a que los fragmentos que se mueven entre la memoria y el disco son de un tamaño igual y prefijado, es posible desarrollar algoritmos de gestión de la memoria más sofisticados que exploten el comportamiento de los programas, como veremos más adelante. La segmentación sí es visible al programador y tiene los beneficios que hemos visto anteriormente, incluyendo la posibilidad de manejar estructuras de datos que crecen, modularidad, y dar soporte a la compartición y a la protección.

Para combinar las ventajas de ambos, algunos sistemas por medio del hardware del procesador y del soporte del sistema operativo son capaces de proporcionar ambos.

En un sistema combinado de paginación/segmentación, el espacio de direcciones del usuario se divide en un número de segmentos, a discreción del programador. Cada segmento es, por su parte, dividido en un número de páginas de tamaño fijo, que son del tamaño de los marcos de la memoria principal. Si un segmento tiene longitud inferior a una página, el segmento ocupará únicamente una página.

Desde el punto de vista del programador, una dirección lógica sigue conteniendo un ***número de segmento y un desplazamiento dentro de dicho segmento***.

Desde el *punto de vista del sistema*, el desplazamiento dentro del segmento es visto como un ***número de página y un desplazamiento dentro de la página*** incluida en el segmento



Traducción de direcciones en un sistema con segmentación / paginación